HIS Microservices — Complete API Implementation Guide

Title: HIS Microservices — 10 REST APIs with Full Implementation
Services: Eligibility Service (5 APIs) · Notice Service (5 APIs)
Technology: Spring Boot 3.2 · Spring Batch 5 · JPA · MySQL · Kafka
Date: March 2026

API Summary — Both Services

#	Service	Method	Endpoint	Purpose
1	Eligibility	`POST`	`/api/eligibility/check`	Check plan eligibility
2	Eligibility	`GET`	`/api/eligibility/plans`	List all active plans
3	Eligibility	`GET`	`/api/eligibility/plans/{id}`	Get plan by ID
4	Eligibility	`PUT`	`/api/eligibility/plans/{id}`	Update a plan
5	Eligibility	`DELETE`	`/api/eligibility/plans/{id}`	Soft delete a plan
6	Notice	`POST`	`/api/notices/run-batch`	Trigger notice batch
7	Notice	`GET`	`/api/notices/{id}`	Get notice by ID
8	Notice	`GET`	`/api/notices/status/{status}`	List notices by status
9	Notice	`POST`	`/api/notices/retry/{id}`	Retry a failed notice
10	Notice	`GET`	`/api/notices/download/{id}`	Download notice PDF

PART 1 — ELIGIBILITY SERVICE (Port 8080)

Entities

EligibilityPlan.java

```java
package com.eligibility.entity;

import jakarta.persistence.*;
import lombok.*;
import java.time.LocalDate;

@Entity
@Table(name = "eligibility_plan")
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class EligibilityPlan {
```

```java
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String planName;

    @Column(nullable = false)
    private String memberName;

    @Column(nullable = false)
    private Integer age;

    @Column(nullable = false)
    private Double income;

    @Column(nullable = false)
    private String gender;

    @Column(nullable = false)
    private LocalDate planStartDate;

    @Column(nullable = false)
    private LocalDate planEndDate;

    @Column(nullable = false)
    @Builder.Default
    private Boolean activeSw = true;
}
```

EligibilityRule.java

```java
package com.eligibility.entity;

import jakarta.persistence.*;
import lombok.*;

@Entity
@Table(name = "eligibility_rule")
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class EligibilityRule {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String planName;

    @Column(nullable = false)
```

```
    private Integer minAge;

    @Column(nullable = false)
    private Integer maxAge;

    @Column(nullable = false)
    private Double maxIncome;

    @Column(nullable = false)
    @Builder.Default
    private Boolean activeSw = true;
}
```

DTOs

EligibilityRequest.java

```
package com.eligibility.dto;

import jakarta.validation.constraints.*;
import lombok.*;
import java.time.LocalDate;

@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class EligibilityRequest {

    @NotBlank(message = "Plan name is required")
    private String planName;

    @NotBlank(message = "Member name is required")
    private String memberName;

    @NotNull(message = "Age is required")
    @Min(value = 0, message = "Age must be non-negative")
    @Max(value = 150, message = "Age must be realistic")
    private Integer age;

    @NotNull(message = "Income is required")
    @DecimalMin(value = "0.0", message = "Income must be non-negative")
    private Double income;

    @NotBlank(message = "Gender is required")
    private String gender;

    @NotNull(message = "Plan start date is required")
    private LocalDate planStartDate;

    @NotNull(message = "Plan end date is required")
    private LocalDate planEndDate;
```

```
    @Builder.Default
    private Boolean activeSw = true;
}
```

EligibilityResponse.java

```
package com.eligibility.dto;

import lombok.*;
import java.time.LocalDate;
import java.util.List;

@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class EligibilityResponse {

    private Long planId;
    private String planName;
    private String memberName;
    private Integer age;
    private Double income;
    private String gender;
    private LocalDate planStartDate;
    private LocalDate planEndDate;
    private Boolean activeSw;
    private boolean eligible;
    private List<String> messages;
}
```

Repositories

EligibilityPlanRepository.java

```
package com.eligibility.repository;

import com.eligibility.entity.EligibilityPlan;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import java.util.List;
import java.util.Optional;

@Repository
public interface EligibilityPlanRepository extends JpaRepository<EligibilityPlan, Long> {

    List<EligibilityPlan> findAllByActiveSw(Boolean activeSw);

    Optional<EligibilityPlan> findByIdAndActiveSw(Long id, Boolean activeSw);
}
```

EligibilityRuleRepository.java

```java
package com.eligibility.repository;

import com.eligibility.entity.EligibilityRule;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import java.util.Optional;

@Repository
public interface EligibilityRuleRepository extends JpaRepository<EligibilityRule, Long> {

    Optional<EligibilityRule> findByPlanNameAndActiveSw(String planName, Boolean activeSw);
}
```

Strategy Interface & Factory

EligibilityStrategy.java

```java
package com.eligibility.strategy;

import com.eligibility.dto.EligibilityRequest;
import com.eligibility.dto.EligibilityResponse;

public interface EligibilityStrategy {
    EligibilityResponse check(EligibilityRequest request);
    String getPlanName();
}
```

EligibilityFactory.java

```java
package com.eligibility.factory;

import com.eligibility.exception.StrategyNotFoundException;
import com.eligibility.strategy.EligibilityStrategy;
import org.springframework.stereotype.Component;
import java.util.List;
import java.util.Map;
import java.util.function.Function;
import java.util.stream.Collectors;

@Component
public class EligibilityFactory {

    private final Map<String, EligibilityStrategy> strategyMap;

    public EligibilityFactory(List<EligibilityStrategy> strategies) {
        this.strategyMap = strategies.stream()
```

```
                .collect(Collectors.toMap(
                        EligibilityStrategy::getPlanName,
                        Function.identity()));
    }

    public EligibilityStrategy getStrategy(String planName) {
        EligibilityStrategy strategy = strategyMap.get(planName.toUpperCase());
        if (strategy == null) {
            throw new StrategyNotFoundException(
                "No eligibility strategy found for plan: " + planName);
        }
        return strategy;
    }

    public java.util.Set<String> getRegisteredPlans() {
        return strategyMap.keySet();
    }
}
```

Strategy Implementations (4 Plans)

SNAPEligibilityStrategy.java

```
package com.eligibility.strategy;

import com.eligibility.dto.EligibilityRequest;
import com.eligibility.dto.EligibilityResponse;
import com.eligibility.entity.EligibilityRule;
import com.eligibility.exception.ValidationException;
import com.eligibility.repository.EligibilityRuleRepository;
import org.springframework.stereotype.Component;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

@Component
public class SNAPEligibilityStrategy implements EligibilityStrategy {

    private static final double SNAP_FPL_PERCENTAGE = 1.30;
    private static final int SNAP_MIN_AGE = 18;

    private final EligibilityRuleRepository eligibilityRuleRepository;

    public SNAPEligibilityStrategy(EligibilityRuleRepository eligibilityRuleRepository) {
        this.eligibilityRuleRepository = eligibilityRuleRepository;
    }

    @Override
    public String getPlanName() { return "SNAP"; }
```

```java
    @Override
    public EligibilityResponse check(EligibilityRequest request) {
        List<String> messages = new ArrayList<>();

        EligibilityRule rule = eligibilityRuleRepository
                .findByPlanNameAndActiveSw(request.getPlanName().toUpperCase(), true)
                .orElseThrow(() -> new ValidationException(
                        "No active eligibility rule found for plan: " +
request.getPlanName()));

        boolean eligible = true;

        // Age range (DB-driven)
        if (request.getAge() < rule.getMinAge() || request.getAge() > rule.getMaxAge()) {
            messages.add(String.format("Age %d is outside allowed range %d-%d",
                    request.getAge(), rule.getMinAge(), rule.getMaxAge()));
            eligible = false;
        }

        // SNAP-specific: must be 18+
        if (request.getAge() < SNAP_MIN_AGE) {
            messages.add("SNAP requires applicant to be at least 18 years old");
            eligible = false;
        }

        // Income (130% FPL)
        double incomeThreshold = rule.getMaxIncome() * SNAP_FPL_PERCENTAGE;
        if (request.getIncome() > incomeThreshold) {
            messages.add(String.format("SNAP income exceeds 130%% FPL threshold of $%.2f",
                    incomeThreshold));
            eligible = false;
        }

        // Plan active status
        if (request.getActiveSw() == null || !request.getActiveSw()) {
            messages.add("Plan is not active");
            eligible = false;
        }

        // Plan date validation
        LocalDate today = LocalDate.now();
        if (request.getPlanStartDate() != null && request.getPlanStartDate().isAfter(today))
{
            messages.add("Plan has not started yet");
            eligible = false;
        }
        if (request.getPlanEndDate() != null && request.getPlanEndDate().isBefore(today)) {
            messages.add("Plan has already ended");
            eligible = false;
        }

        if (eligible) messages.add("Applicant is ELIGIBLE for SNAP");
```

```
        return EligibilityResponse.builder()
                .planName(request.getPlanName()).memberName(request.getMemberName())

.age(request.getAge()).income(request.getIncome()).gender(request.getGender())

.planStartDate(request.getPlanStartDate()).planEndDate(request.getPlanEndDate())
                .activeSw(request.getActiveSw()).eligible(eligible).messages(messages)
                .build();
    }
}
```

MedicaidEligibilityStrategy.java

```
package com.eligibility.strategy;

// Same structure as SNAP — key differences:
@Component
public class MedicaidEligibilityStrategy implements EligibilityStrategy {

    private static final double MEDICAID_FPL_PERCENTAGE = 1.38;

    private final EligibilityRuleRepository eligibilityRuleRepository;

    public MedicaidEligibilityStrategy(EligibilityRuleRepository repo) {
        this.eligibilityRuleRepository = repo;
    }

    @Override
    public String getPlanName() { return "MEDICAID"; }

    @Override
    public EligibilityResponse check(EligibilityRequest request) {
        List<String> messages = new ArrayList<>();
        EligibilityRule rule = eligibilityRuleRepository
                .findByPlanNameAndActiveSw(request.getPlanName().toUpperCase(), true)
                .orElseThrow(() -> new ValidationException("No active rule for: " +
request.getPlanName()));

        boolean eligible = true;

        // Age range (0-64 from DB)
        if (request.getAge() < rule.getMinAge() || request.getAge() > rule.getMaxAge()) {
            messages.add(String.format("Age %d outside range %d-%d",
                    request.getAge(), rule.getMinAge(), rule.getMaxAge()));
            eligible = false;
        }

        // Income (138% FPL)
        double threshold = rule.getMaxIncome() * MEDICAID_FPL_PERCENTAGE;
        if (request.getIncome() > threshold) {
```

```
            messages.add(String.format("Medicaid income exceeds 138%% FPL threshold of
$%.2f", threshold));
            eligible = false;
        }

        // Active + Date checks (same pattern as SNAP)
        if (request.getActiveSw() == null || !request.getActiveSw()) {
            messages.add("Plan is not active"); eligible = false;
        }
        LocalDate today = LocalDate.now();
        if (request.getPlanStartDate() != null && request.getPlanStartDate().isAfter(today))
{
            messages.add("Plan has not started yet"); eligible = false;
        }
        if (request.getPlanEndDate() != null && request.getPlanEndDate().isBefore(today)) {
            messages.add("Plan has already ended"); eligible = false;
        }

        if (eligible) messages.add("Applicant is ELIGIBLE for MEDICAID");

        return EligibilityResponse.builder()
                .planName(request.getPlanName()).memberName(request.getMemberName())

.age(request.getAge()).income(request.getIncome()).gender(request.getGender())

.planStartDate(request.getPlanStartDate()).planEndDate(request.getPlanEndDate())
                .activeSw(request.getActiveSw()).eligible(eligible).messages(messages)
                .build();
    }
}
```

MedicareEligibilityStrategy.java

```
@Component
public class MedicareEligibilityStrategy implements EligibilityStrategy {

    private static final int MEDICARE_MIN_AGE = 65;
    private final EligibilityRuleRepository eligibilityRuleRepository;

    public MedicareEligibilityStrategy(EligibilityRuleRepository repo) {
        this.eligibilityRuleRepository = repo;
    }

    @Override public String getPlanName() { return "MEDICARE"; }

    @Override
    public EligibilityResponse check(EligibilityRequest request) {
        List<String> messages = new ArrayList<>();
        EligibilityRule rule = eligibilityRuleRepository
                .findByPlanNameAndActiveSw(request.getPlanName().toUpperCase(), true)
                .orElseThrow(() -> new ValidationException("No active rule for: " +
```

```
request.getPlanName()));

        boolean eligible = true;

        // Medicare: must be 65+
        if (request.getAge() < MEDICARE_MIN_AGE) {
            messages.add(String.format("Medicare requires age >= %d, applicant is %d",
                    MEDICARE_MIN_AGE, request.getAge()));
            eligible = false;
        }

        // Income: direct max comparison
        if (request.getIncome() > rule.getMaxIncome()) {
            messages.add(String.format("Income $%.2f exceeds max $%.2f",
                    request.getIncome(), rule.getMaxIncome()));
            eligible = false;
        }

        // Active + Date checks
        if (request.getActiveSw() == null || !request.getActiveSw()) {
            messages.add("Plan is not active"); eligible = false;
        }
        LocalDate today = LocalDate.now();
        if (request.getPlanStartDate() != null && request.getPlanStartDate().isAfter(today))
{
            messages.add("Plan has not started yet"); eligible = false;
        }
        if (request.getPlanEndDate() != null && request.getPlanEndDate().isBefore(today)) {
            messages.add("Plan has already ended"); eligible = false;
        }

        if (eligible) messages.add("Applicant is ELIGIBLE for MEDICARE");

        return EligibilityResponse.builder()
                .planName(request.getPlanName()).memberName(request.getMemberName())

.age(request.getAge()).income(request.getIncome()).gender(request.getGender())

.planStartDate(request.getPlanStartDate()).planEndDate(request.getPlanEndDate())
                .activeSw(request.getActiveSw()).eligible(eligible).messages(messages)
                .build();
    }
}
```

QHPEligibilityStrategy.java

```
@Component
public class QHPEligibilityStrategy implements EligibilityStrategy {

    private static final int QHP_MAX_AGE = 64;
    private final EligibilityRuleRepository eligibilityRuleRepository;
```

```java
    public QHPEligibilityStrategy(EligibilityRuleRepository repo) {
        this.eligibilityRuleRepository = repo;
    }

    @Override public String getPlanName() { return "QHP"; }

    @Override
    public EligibilityResponse check(EligibilityRequest request) {
        List<String> messages = new ArrayList<>();
        EligibilityRule rule = eligibilityRuleRepository
                .findByPlanNameAndActiveSw(request.getPlanName().toUpperCase(), true)
                .orElseThrow(() -> new ValidationException("No active rule for: " +
request.getPlanName()));

        boolean eligible = true;

        // QHP: must be 64 or younger
        if (request.getAge() > QHP_MAX_AGE) {
            messages.add(String.format("QHP requires age <= %d, applicant is %d",
                    QHP_MAX_AGE, request.getAge()));
            eligible = false;
        }

        // Income: direct max comparison
        if (request.getIncome() > rule.getMaxIncome()) {
            messages.add(String.format("QHP income $%.2f exceeds max $%.2f",
                    request.getIncome(), rule.getMaxIncome()));
            eligible = false;
        }

        // Active + Date checks
        if (request.getActiveSw() == null || !request.getActiveSw()) {
            messages.add("Plan is not active"); eligible = false;
        }
        LocalDate today = LocalDate.now();
        if (request.getPlanStartDate() != null && request.getPlanStartDate().isAfter(today))
{
            messages.add("Plan has not started yet"); eligible = false;
        }
        if (request.getPlanEndDate() != null && request.getPlanEndDate().isBefore(today)) {
            messages.add("Plan has already ended"); eligible = false;
        }

        if (eligible) messages.add("Applicant is ELIGIBLE for QHP");

        return EligibilityResponse.builder()
                .planName(request.getPlanName()).memberName(request.getMemberName())

.age(request.getAge()).income(request.getIncome()).gender(request.getGender())

.planStartDate(request.getPlanStartDate()).planEndDate(request.getPlanEndDate())
```

```
                .activeSw(request.getActiveSw()).eligible(eligible).messages(messages)
                .build();
    }
}
```

Service Layer

EligibilityService.java

```
package com.eligibility.service;

import com.eligibility.dto.EligibilityRequest;
import com.eligibility.dto.EligibilityResponse;
import com.eligibility.entity.EligibilityPlan;
import com.eligibility.exception.PlanNotFoundException;
import com.eligibility.factory.EligibilityFactory;
import com.eligibility.repository.EligibilityPlanRepository;
import com.eligibility.strategy.EligibilityStrategy;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.util.List;
import java.util.stream.Collectors;

@Service
@Transactional
public class EligibilityService {

    private final EligibilityPlanRepository planRepository;
    private final EligibilityFactory eligibilityFactory;

    public EligibilityService(EligibilityPlanRepository planRepository,
                              EligibilityFactory eligibilityFactory) {
        this.planRepository = planRepository;
        this.eligibilityFactory = eligibilityFactory;
    }

    // ── API 1: Check Eligibility ──────────────────────────────────────
    public EligibilityResponse checkEligibility(EligibilityRequest request) {
        EligibilityPlan plan = EligibilityPlan.builder()
                .planName(request.getPlanName().toUpperCase())
                .memberName(request.getMemberName())
                .age(request.getAge())
                .income(request.getIncome())
                .gender(request.getGender())
                .planStartDate(request.getPlanStartDate())
                .planEndDate(request.getPlanEndDate())
                .activeSw(request.getActiveSw() != null ? request.getActiveSw() : true)
                .build();
        plan = planRepository.save(plan);
```

```
        EligibilityStrategy strategy =
eligibilityFactory.getStrategy(request.getPlanName());
        EligibilityResponse response = strategy.check(request);
        response.setPlanId(plan.getId());
        return response;
    }

    // ── API 2: Get All Active Plans ──────────────────────────────
    @Transactional(readOnly = true)
    public List<EligibilityResponse> getAllActivePlans() {
        return planRepository.findAllByActiveSw(true).stream()
                .map(this::mapToResponse)
                .collect(Collectors.toList());
    }

    // ── API 3: Get Plan by ID ────────────────────────────────────
    @Transactional(readOnly = true)
    public EligibilityResponse getPlanById(Long id) {
        EligibilityPlan plan = planRepository.findByIdAndActiveSw(id, true)
                .orElseThrow(() -> new PlanNotFoundException("Plan not found with id: " +
id));
        return mapToResponse(plan);
    }

    // ── API 4: Update Plan ───────────────────────────────────────
    public EligibilityResponse updatePlan(Long id, EligibilityRequest request) {
        EligibilityPlan plan = planRepository.findByIdAndActiveSw(id, true)
                .orElseThrow(() -> new PlanNotFoundException("Plan not found with id: " +
id));

        plan.setPlanName(request.getPlanName().toUpperCase());
        plan.setMemberName(request.getMemberName());
        plan.setAge(request.getAge());
        plan.setIncome(request.getIncome());
        plan.setGender(request.getGender());
        plan.setPlanStartDate(request.getPlanStartDate());
        plan.setPlanEndDate(request.getPlanEndDate());
        if (request.getActiveSw() != null) plan.setActiveSw(request.getActiveSw());

        plan = planRepository.save(plan);
        return mapToResponse(plan);
    }

    // ── API 5: Soft Delete Plan ──────────────────────────────────
    public void softDeletePlan(Long id) {
        EligibilityPlan plan = planRepository.findByIdAndActiveSw(id, true)
                .orElseThrow(() -> new PlanNotFoundException("Plan not found with id: " +
id));
        plan.setActiveSw(false);
        planRepository.save(plan);
    }
```

```java
    // ── Mapper ──────────────────────────────────────────────────────
    private EligibilityResponse mapToResponse(EligibilityPlan plan) {
        return EligibilityResponse.builder()
                .planId(plan.getId()).planName(plan.getPlanName())
                .memberName(plan.getMemberName()).age(plan.getAge())
                .income(plan.getIncome()).gender(plan.getGender())
                .planStartDate(plan.getPlanStartDate()).planEndDate(plan.getPlanEndDate())
                .activeSw(plan.getActiveSw())
                .build();
    }
}
```

Controller Layer

EligibilityController.java

```java
package com.eligibility.controller;

import com.eligibility.dto.EligibilityRequest;
import com.eligibility.dto.EligibilityResponse;
import com.eligibility.service.EligibilityService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/eligibility")
@Tag(name = "Eligibility Controller",
     description = "Endpoints for eligibility determination and plan management")
public class EligibilityController {

    private final EligibilityService eligibilityService;

    public EligibilityController(EligibilityService eligibilityService) {
        this.eligibilityService = eligibilityService;
    }

    // ─── API 1: Check Eligibility ──────────────────────────────────
    @PostMapping("/check")
    @Operation(summary = "Check eligibility",
               description = "Determines eligibility for the specified plan")
    public ResponseEntity<EligibilityResponse> checkEligibility(
            @Valid @RequestBody EligibilityRequest request) {
        return new ResponseEntity<>(
                eligibilityService.checkEligibility(request), HttpStatus.OK);
    }
```

```java
    // ── API 2: Get All Active Plans ──────────────────────────────
    @GetMapping("/plans")
    @Operation(summary = "Get all active plans")
    public ResponseEntity<List<EligibilityResponse>> getAllPlans() {
        return new ResponseEntity<>(
                eligibilityService.getAllActivePlans(), HttpStatus.OK);
    }

    // ── API 3: Get Plan by ID ────────────────────────────────────
    @GetMapping("/plans/{id}")
    @Operation(summary = "Get plan by ID")
    public ResponseEntity<EligibilityResponse> getPlanById(@PathVariable Long id) {
        return new ResponseEntity<>(
                eligibilityService.getPlanById(id), HttpStatus.OK);
    }

    // ── API 4: Update Plan ───────────────────────────────────────
    @PutMapping("/plans/{id}")
    @Operation(summary = "Update plan")
    public ResponseEntity<EligibilityResponse> updatePlan(
            @PathVariable Long id, @Valid @RequestBody EligibilityRequest request) {
        return new ResponseEntity<>(
                eligibilityService.updatePlan(id, request), HttpStatus.OK);
    }

    // ── API 5: Soft Delete Plan ──────────────────────────────────
    @DeleteMapping("/plans/{id}")
    @Operation(summary = "Soft delete plan")
    public ResponseEntity<String> softDeletePlan(@PathVariable Long id) {
        eligibilityService.softDeletePlan(id);
        return new ResponseEntity<>("Plan soft-deleted successfully", HttpStatus.OK);
    }
}
```

Exception Handling

Exceptions & GlobalExceptionHandler

```java
// ── PlanNotFoundException.java ──
public class PlanNotFoundException extends RuntimeException {
    public PlanNotFoundException(String message) { super(message); }
}

// ── StrategyNotFoundException.java ──
public class StrategyNotFoundException extends RuntimeException {
    public StrategyNotFoundException(String message) { super(message); }
}

// ── ValidationException.java ──
```

```java
public class ValidationException extends RuntimeException {
    public ValidationException(String message) { super(message); }
}

// ── ErrorResponse.java ──
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class ErrorResponse {
    private int status;
    private String message;
    private LocalDateTime timestamp;
}

// ── GlobalExceptionHandler.java ──
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(PlanNotFoundException.class)
    public ResponseEntity<ErrorResponse> handlePlanNotFound(PlanNotFoundException ex) {
        return buildResponse(HttpStatus.NOT_FOUND, ex.getMessage());
    }

    @ExceptionHandler(StrategyNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleStrategyNotFound(StrategyNotFoundException
ex) {
        return buildResponse(HttpStatus.BAD_REQUEST, ex.getMessage());
    }

    @ExceptionHandler(ValidationException.class)
    public ResponseEntity<ErrorResponse> handleValidation(ValidationException ex) {
        return buildResponse(HttpStatus.BAD_REQUEST, ex.getMessage());
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse>
handleMethodArgument(MethodArgumentNotValidException ex) {
        String errors = ex.getBindingResult().getFieldErrors().stream()
                .map(e -> e.getField() + ": " + e.getDefaultMessage())
                .collect(Collectors.joining(", "));
        return buildResponse(HttpStatus.BAD_REQUEST, errors);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGeneric(Exception ex) {
        return buildResponse(HttpStatus.INTERNAL_SERVER_ERROR,
                "Unexpected error: " + ex.getMessage());
    }

    private ResponseEntity<ErrorResponse> buildResponse(HttpStatus status, String msg) {
        return new ResponseEntity<>(ErrorResponse.builder()
                .status(status.value()).message(msg)
                .timestamp(LocalDateTime.now()).build(), status);
```

```
    }
}
```

Configuration — application.yml

```yaml
server:
  port: 8080

spring:
  application:
    name: eligibility-service
  datasource:
    url: jdbc:mysql://localhost:3306/test?
createDatabaseIfNotExist=true&useSSL=false&allowPublicKeyRetrieval=true
    driver-class-name: com.mysql.cj.jdbc.Driver
    username: root
    password: root
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
    properties:
      hibernate:
        dialect: org.hibernate.dialect.MySQLDialect
    defer-datasource-initialization: true
  sql:
    init:
      mode: always

springdoc:
  api-docs:
    path: /api-docs
  swagger-ui:
    path: /swagger-ui.html
```

Eligibility Service — API Request/Response Examples

API 1: POST `/api/eligibility/check`

Request:

```json
{
    "planName": "SNAP",
    "memberName": "John Doe",
    "age": 30,
    "income": 2000.00,
    "gender": "Male",
    "planStartDate": "2026-01-01",
    "planEndDate": "2026-12-31",
```

```
    "activeSw": true
}
```

Response (200 OK):

```
{
    "planId": 1,
    "planName": "SNAP",
    "memberName": "John Doe",
    "age": 30,
    "income": 2000.00,
    "gender": "Male",
    "planStartDate": "2026-01-01",
    "planEndDate": "2026-12-31",
    "activeSw": true,
    "eligible": true,
    "messages": ["Applicant is ELIGIBLE for SNAP"]
}
```

API 2: GET `/api/eligibility/plans`

Response (200 OK):

```
[
    { "planId": 1, "planName": "SNAP", "memberName": "John Doe", ... },
    { "planId": 2, "planName": "MEDICAID", "memberName": "Jane Doe", ... }
]
```

API 3: GET `/api/eligibility/plans/1`

Response (200 OK): Single `EligibilityResponse` object
Response (404): `{ "status": 404, "message": "Plan not found with id: 1" }`

API 4: PUT `/api/eligibility/plans/1`

Request: Same as check request body
Response (200 OK): Updated `EligibilityResponse`

API 5: DELETE `/api/eligibility/plans/1`

Response (200 OK): `"Plan soft-deleted successfully"`
No physical delete — sets activeSw = false

PART 2 — NOTICE SERVICE (Port 8082)

Entities

Notice.java

```java
package com.notice.entity;

import com.notice.enums.NoticeStatus;
import com.notice.enums.NoticeType;
import jakarta.persistence.*;
import lombok.*;
import java.time.LocalDateTime;
import java.util.List;

@Entity
@Table(name = "notice", indexes = {
    @Index(name = "idx_notice_status", columnList = "status"),
    @Index(name = "idx_notice_batch_id", columnList = "batchId"),
    @Index(name = "idx_notice_member", columnList = "memberId")
})
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class Notice {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String noticeNumber;

    @Column(nullable = false)
    private Long memberId;

    @Column(nullable = false)
    private String memberName;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private NoticeType noticeType;

    @Column(nullable = false)
    private String planName;

    @Column(columnDefinition = "TEXT")
    private String contentSummary;

    private String pdfFilePath;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private NoticeStatus status;

    @Column(nullable = false)
    @Builder.Default
    private Integer retryCount = 0;
```

```java
    private String batchId;

    private LocalDateTime generatedAt;
    private LocalDateTime sentAt;

    @Column(nullable = false, updatable = false)
    private LocalDateTime createdAt;

    @Column(nullable = false)
    private LocalDateTime updatedAt;

    @Column(nullable = false)
    @Builder.Default
    private Boolean activeSw = true;

    @OneToMany(mappedBy = "notice", cascade = CascadeType.ALL, fetch = FetchType.LAZY)
    private List<NoticeHistory> history;

    @PrePersist
    protected void onCreate() {
        createdAt = LocalDateTime.now();
        updatedAt = LocalDateTime.now();
    }

    @PreUpdate
    protected void onUpdate() {
        updatedAt = LocalDateTime.now();
    }
}
```

NoticeHistory.java

```java
package com.notice.entity;

import com.notice.enums.NoticeStatus;
import jakarta.persistence.*;
import lombok.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "notice_history")
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class NoticeHistory {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "notice_id", nullable = false)
    private Notice notice;
```

```java
    @Enumerated(EnumType.STRING)
    private NoticeStatus previousStatus;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private NoticeStatus newStatus;

    private String changedBy;
    private String changeReason;

    @Column(nullable = false)
    private LocalDateTime changedAt;

    @PrePersist
    protected void onCreate() {
        changedAt = LocalDateTime.now();
    }
}
```

NoticeBatchExecution.java

```java
package com.notice.entity;

import jakarta.persistence.*;
import lombok.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "notice_batch_execution")
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class NoticeBatchExecution {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String batchId;

    @Column(nullable = false)
    private String jobName;

    private LocalDateTime startTime;
    private LocalDateTime endTime;

    @Column(nullable = false)
    private String status;  // STARTED, COMPLETED, FAILED

    @Builder.Default
    private Integer totalRecords = 0;
```

```java
    @Builder.Default
    private Integer successCount = 0;
    @Builder.Default
    private Integer failureCount = 0;
    @Builder.Default
    private Integer skipCount = 0;

    @Column(nullable = false)
    private String triggerType;  // SCHEDULED, MANUAL

    private String triggeredBy;

    @Column(columnDefinition = "TEXT")
    private String errorMessage;
}
```

Enums

```java
// ── NoticeStatus.java ──
package com.notice.enums;

public enum NoticeStatus {
    PENDING, GENERATED, SENT, DELIVERED, FAILED
}

// ── NoticeType.java ──
package com.notice.enums;

public enum NoticeType {
    ELIGIBILITY, RENEWAL, TERMINATION, BENEFIT_CHANGE
}
```

DTOs

```java
// ── NoticeResponse.java ──
package com.notice.dto;

import com.notice.enums.NoticeStatus;
import com.notice.enums.NoticeType;
import lombok.*;
import java.time.LocalDateTime;

@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class NoticeResponse {
    private Long id;
    private String noticeNumber;
    private Long memberId;
    private String memberName;
    private NoticeType noticeType;
```

```java
    private String planName;
    private String contentSummary;
    private NoticeStatus status;
    private Integer retryCount;
    private String batchId;
    private LocalDateTime generatedAt;
    private LocalDateTime sentAt;
    private LocalDateTime createdAt;
}

// ── BatchExecutionResponse.java ──
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class BatchExecutionResponse {
    private String batchId;
    private String status;
    private String message;
    private LocalDateTime startTime;
    private LocalDateTime endTime;
    private Integer totalRecords;
    private Integer successCount;
    private Integer failureCount;
    private Integer skipCount;
}
```

Repositories

```java
// ── NoticeRepository.java ──
package com.notice.repository;

import com.notice.entity.Notice;
import com.notice.enums.NoticeStatus;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import java.util.List;
import java.util.Optional;

@Repository
public interface NoticeRepository extends JpaRepository<Notice, Long> {

    Optional<Notice> findByIdAndActiveSw(Long id, Boolean activeSw);

    List<Notice> findAllByStatusAndActiveSw(NoticeStatus status, Boolean activeSw);

    Page<Notice> findAllByStatus(NoticeStatus status, Pageable pageable);

    List<Notice> findAllByBatchId(String batchId);
}
```

```java
// ── NoticeHistoryRepository.java ──
@Repository
public interface NoticeHistoryRepository extends JpaRepository<NoticeHistory, Long> {
    List<NoticeHistory> findAllByNoticeIdOrderByChangedAtDesc(Long noticeId);
}

// ── BatchExecutionRepository.java ──
@Repository
public interface BatchExecutionRepository extends JpaRepository<NoticeBatchExecution, Long>
{
    Optional<NoticeBatchExecution> findByBatchId(String batchId);
}
```

Service Layer

NoticeService.java (Interface)

```java
package com.notice.service;

import com.notice.dto.BatchExecutionResponse;
import com.notice.dto.NoticeResponse;
import com.notice.enums.NoticeStatus;
import java.util.List;

public interface NoticeService {
    BatchExecutionResponse triggerBatch(String triggeredBy);
    NoticeResponse getNoticeById(Long id);
    List<NoticeResponse> getNoticesByStatus(NoticeStatus status);
    NoticeResponse retryNotice(Long id);
    byte[] downloadNoticePdf(Long id);
}
```

NoticeServiceImpl.java

```java
package com.notice.service.impl;

import com.notice.dto.BatchExecutionResponse;
import com.notice.dto.NoticeResponse;
import com.notice.entity.Notice;
import com.notice.entity.NoticeBatchExecution;
import com.notice.entity.NoticeHistory;
import com.notice.enums.NoticeStatus;
import com.notice.exception.NoticeNotFoundException;
import com.notice.mapper.NoticeMapper;
import com.notice.repository.BatchExecutionRepository;
import com.notice.repository.NoticeHistoryRepository;
import com.notice.repository.NoticeRepository;
import com.notice.service.NoticeService;
import org.springframework.batch.core.Job;
```

```java
import org.springframework.batch.core.JobParameters;
import org.springframework.batch.core.JobParametersBuilder;
import org.springframework.batch.core.launch.JobLauncher;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.time.LocalDateTime;
import java.util.List;
import java.util.UUID;
import java.util.stream.Collectors;

@Service
@Transactional
public class NoticeServiceImpl implements NoticeService {

    private final NoticeRepository noticeRepository;
    private final NoticeHistoryRepository historyRepository;
    private final BatchExecutionRepository batchRepo;
    private final JobLauncher jobLauncher;
    private final Job noticeGenerationJob;

    public NoticeServiceImpl(NoticeRepository noticeRepository,
                             NoticeHistoryRepository historyRepository,
                             BatchExecutionRepository batchRepo,
                             JobLauncher jobLauncher,
                             Job noticeGenerationJob) {
        this.noticeRepository = noticeRepository;
        this.historyRepository = historyRepository;
        this.batchRepo = batchRepo;
        this.jobLauncher = jobLauncher;
        this.noticeGenerationJob = noticeGenerationJob;
    }

    // ── API 6: Trigger Batch ──────────────────────────────────────────
    @Override
    public BatchExecutionResponse triggerBatch(String triggeredBy) {
        String batchId = UUID.randomUUID().toString();

        NoticeBatchExecution execution = NoticeBatchExecution.builder()
                .batchId(batchId).jobName("noticeGenerationJob")
                .startTime(LocalDateTime.now()).status("STARTED")
                .triggerType("MANUAL").triggeredBy(triggeredBy)
                .build();
        batchRepo.save(execution);

        try {
            JobParameters params = new JobParametersBuilder()
                    .addString("batchId", batchId)
                    .addLocalDateTime("triggerTime", LocalDateTime.now())
                    .toJobParameters();
```

```java
        jobLauncher.run(noticeGenerationJob, params);
    } catch (Exception e) {
        execution.setStatus("FAILED");
        execution.setErrorMessage(e.getMessage());
        execution.setEndTime(LocalDateTime.now());
        batchRepo.save(execution);
    }

    return BatchExecutionResponse.builder()
            .batchId(batchId).status("STARTED")
            .message("Batch triggered successfully")
            .startTime(execution.getStartTime())
            .build();
}

// ── API 7: Get Notice by ID ──────────────────────────────────
@Override
@Transactional(readOnly = true)
public NoticeResponse getNoticeById(Long id) {
    Notice notice = noticeRepository.findByIdAndActiveSw(id, true)
            .orElseThrow(() -> new NoticeNotFoundException(
                    "Notice not found with id: " + id));
    return NoticeMapper.toResponse(notice);
}

// ── API 8: Get Notices by Status ─────────────────────────────
@Override
@Transactional(readOnly = true)
public List<NoticeResponse> getNoticesByStatus(NoticeStatus status) {
    return noticeRepository.findAllByStatusAndActiveSw(status, true)
            .stream()
            .map(NoticeMapper::toResponse)
            .collect(Collectors.toList());
}

// ── API 9: Retry Failed Notice ───────────────────────────────
@Override
public NoticeResponse retryNotice(Long id) {
    Notice notice = noticeRepository.findByIdAndActiveSw(id, true)
            .orElseThrow(() -> new NoticeNotFoundException(
                    "Notice not found with id: " + id));

    if (notice.getStatus() != NoticeStatus.FAILED) {
        throw new IllegalStateException(
            "Cannot retry notice with status: " + notice.getStatus());
    }

    // Save history
    NoticeHistory history = NoticeHistory.builder()
            .notice(notice)
            .previousStatus(NoticeStatus.FAILED)
            .newStatus(NoticeStatus.PENDING)
```

```
                .changedBy("SYSTEM")
                .changeReason("Manual retry triggered")
                .build();
        historyRepository.save(history);

        // Reset to PENDING for next batch
        notice.setStatus(NoticeStatus.PENDING);
        notice.setRetryCount(notice.getRetryCount() + 1);
        noticeRepository.save(notice);

        return NoticeMapper.toResponse(notice);
    }

    // ── API 10: Download Notice PDF ──────────────────────────────
    @Override
    @Transactional(readOnly = true)
    public byte[] downloadNoticePdf(Long id) {
        Notice notice = noticeRepository.findByIdAndActiveSw(id, true)
                .orElseThrow(() -> new NoticeNotFoundException(
                        "Notice not found with id: " + id));

        if (notice.getStatus() == NoticeStatus.PENDING) {
            throw new IllegalStateException("Notice has not been generated yet");
        }

        if (notice.getPdfFilePath() == null) {
            throw new NoticeNotFoundException("PDF not found for notice: " + id);
        }

        try {
            return Files.readAllBytes(Path.of(notice.getPdfFilePath()));
        } catch (IOException e) {
            throw new RuntimeException("Failed to read PDF file", e);
        }
    }
}
```

Controller Layer

NoticeController.java

```
package com.notice.controller;

import com.notice.dto.BatchExecutionResponse;
import com.notice.dto.NoticeResponse;
import com.notice.enums.NoticeStatus;
import com.notice.service.NoticeService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.tags.Tag;
import org.springframework.http.*;
```

```java
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/notices")
@Tag(name = "Notice Controller",
     description = "Endpoints for notice generation, management, and download")
public class NoticeController {

    private final NoticeService noticeService;

    public NoticeController(NoticeService noticeService) {
        this.noticeService = noticeService;
    }

    // ── API 6: Trigger Batch ──────────────────────────────────
    @PostMapping("/run-batch")
    @Operation(summary = "Trigger notice generation batch",
               description = "Manually triggers the Spring Batch job")
    public ResponseEntity<BatchExecutionResponse> runBatch() {
        BatchExecutionResponse response = noticeService.triggerBatch("ADMIN");
        return new ResponseEntity<>(response, HttpStatus.OK);
    }

    // ── API 7: Get Notice by ID ───────────────────────────────
    @GetMapping("/{id}")
    @Operation(summary = "Get notice by ID")
    public ResponseEntity<NoticeResponse> getNoticeById(@PathVariable Long id) {
        return new ResponseEntity<>(noticeService.getNoticeById(id), HttpStatus.OK);
    }

    // ── API 8: Get Notices by Status ──────────────────────────
    @GetMapping("/status/{status}")
    @Operation(summary = "Get all notices by status",
               description = "Filter notices: PENDING, GENERATED, SENT, DELIVERED, FAILED")
    public ResponseEntity<List<NoticeResponse>> getNoticesByStatus(
            @PathVariable NoticeStatus status) {
        return new ResponseEntity<>(
                noticeService.getNoticesByStatus(status), HttpStatus.OK);
    }

    // ── API 9: Retry Failed Notice ────────────────────────────
    @PostMapping("/retry/{id}")
    @Operation(summary = "Retry a failed notice",
               description = "Resets FAILED notice to PENDING for next batch")
    public ResponseEntity<NoticeResponse> retryNotice(@PathVariable Long id) {
        return new ResponseEntity<>(noticeService.retryNotice(id), HttpStatus.OK);
    }

    // ── API 10: Download Notice PDF ───────────────────────────
    @GetMapping("/download/{id}")
    @Operation(summary = "Download notice as PDF")
```

```java
    public ResponseEntity<byte[]> downloadNoticePdf(@PathVariable Long id) {
        byte[] pdfBytes = noticeService.downloadNoticePdf(id);

        HttpHeaders headers = new HttpHeaders();
        headers.setContentType(MediaType.APPLICATION_PDF);
        headers.setContentDisposition(
                ContentDisposition.attachment()
                        .filename("notice-" + id + ".pdf")
                        .build());

        return new ResponseEntity<>(pdfBytes, headers, HttpStatus.OK);
    }
}
```

Mapper

```java
package com.notice.mapper;

import com.notice.dto.NoticeResponse;
import com.notice.entity.Notice;

public class NoticeMapper {

    public static NoticeResponse toResponse(Notice notice) {
        return NoticeResponse.builder()
                .id(notice.getId())
                .noticeNumber(notice.getNoticeNumber())
                .memberId(notice.getMemberId())
                .memberName(notice.getMemberName())
                .noticeType(notice.getNoticeType())
                .planName(notice.getPlanName())
                .contentSummary(notice.getContentSummary())
                .status(notice.getStatus())
                .retryCount(notice.getRetryCount())
                .batchId(notice.getBatchId())
                .generatedAt(notice.getGeneratedAt())
                .sentAt(notice.getSentAt())
                .createdAt(notice.getCreatedAt())
                .build();
    }
}
```

Exception Handling

```java
// ── NoticeNotFoundException.java ──
public class NoticeNotFoundException extends RuntimeException {
    public NoticeNotFoundException(String message) { super(message); }
}
```

```java
// ── BatchExecutionException.java ──
public class BatchExecutionException extends RuntimeException {
    public BatchExecutionException(String message) { super(message); }
}

// ── PdfGenerationException.java ──
public class PdfGenerationException extends RuntimeException {
    public PdfGenerationException(String message, Throwable cause) {
        super(message, cause);
    }
}

// ── GlobalExceptionHandler.java (Notice Service) ──
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(NoticeNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(NoticeNotFoundException ex) {
        return buildResponse(HttpStatus.NOT_FOUND, ex.getMessage());
    }

    @ExceptionHandler(IllegalStateException.class)
    public ResponseEntity<ErrorResponse> handleIllegalState(IllegalStateException ex) {
        return buildResponse(HttpStatus.BAD_REQUEST, ex.getMessage());
    }

    @ExceptionHandler(BatchExecutionException.class)
    public ResponseEntity<ErrorResponse> handleBatch(BatchExecutionException ex) {
        return buildResponse(HttpStatus.INTERNAL_SERVER_ERROR, ex.getMessage());
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGeneric(Exception ex) {
        return buildResponse(HttpStatus.INTERNAL_SERVER_ERROR,
                "Unexpected error: " + ex.getMessage());
    }

    private ResponseEntity<ErrorResponse> buildResponse(HttpStatus status, String msg) {
        return new ResponseEntity<>(ErrorResponse.builder()
                .status(status.value()).message(msg)
                .timestamp(LocalDateTime.now()).build(), status);
    }
}
```

Configuration — application.yml (Notice Service)

```yaml
server:
  port: 8082
```

```yaml
spring:
  application:
    name: notice-service
  datasource:
    url: jdbc:mysql://localhost:3306/notice_db?
createDatabaseIfNotExist=true&useSSL=false&allowPublicKeyRetrieval=true
    driver-class-name: com.mysql.cj.jdbc.Driver
    username: root
    password: root
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
    properties:
      hibernate:
        dialect: org.hibernate.dialect.MySQLDialect
  batch:
    jdbc:
      initialize-schema: always
    job:
      enabled: false
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer

notice:
  pdf:
    output-directory: /notices/pdf
  batch:
    chunk-size: 100
```

Notice Service — API Request/Response Examples

API 6: POST `/api/notices/run-batch`

Request Body: None
Response (200 OK):

```json
{
    "batchId": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
    "status": "STARTED",
    "message": "Batch triggered successfully",
    "startTime": "2026-03-04T20:00:00"
}
```

API 7: GET `/api/notices/42`

Response (200 OK):

```
{
    "id": 42,
    "noticeNumber": "NTC-20260304-00042",
    "memberId": 1001,
    "memberName": "John Doe",
    "noticeType": "ELIGIBILITY",
    "planName": "SNAP",
    "contentSummary": "Your eligibility for SNAP has been approved.",
    "status": "GENERATED",
    "retryCount": 0,
    "batchId": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
    "generatedAt": "2026-03-04T02:05:12",
    "sentAt": null,
    "createdAt": "2026-03-03T10:30:00"
}
```

Response (404): `{ "status": 404, "message": "Notice not found with id: 42" }`

API 8: GET `/api/notices/status/FAILED`

Response (200 OK):

```
[
    {
        "id": 55, "noticeNumber": "NTC-20260304-00055",
        "status": "FAILED", "retryCount": 3, ...
    },
    {
        "id": 78, "noticeNumber": "NTC-20260304-00078",
        "status": "FAILED", "retryCount": 1, ...
    }
]
```

API 9: POST `/api/notices/retry/55`

Response (200 OK):

```
{
    "id": 55,
    "noticeNumber": "NTC-20260304-00055",
    "status": "PENDING",
    "retryCount": 4,
    ...
}
```

Response (400): `{ "message": "Cannot retry notice with status: GENERATED" }`

API 10: GET `/api/notices/download/42`

Response (200 OK): PDF file bytes
Headers: `Content-Type: application/pdf`, `Content-Disposition: attachment; filename="notice-42.pdf"`
Response (400): `{ "message": "Notice has not been generated yet" }`

End of Document